represented as `d` for data and `i` for array index, but they could be declared using any available variable name.

The best way to see this in action is to use our data to create a simple data visualization. We'll keep working with `d3ia.html`, which we created in chapter 1, and which is a simple HTML page with minimal DOM elements and styles. A histogram or bar chart is one of the most simple and effective ways of expressing numerical data broken down by category. We'll avoid the more complex datasets for now and start with a simple array of numbers:

```
[15, 50, 22, 8, 100, 10]
```

If we bind this array to a selection, we can use the values to determine the height of the rectangles (our bars in a bar chart). We need to set a width based on the space available for the chart, and we'll start by setting it to 10 px:

```
d3.select("svg")
  .selectAll("rect")
  .data([15, 50, 22, 8, 100, 10])
  .enter()
  .append("rect")
  .attr("width", 10)
  .attr("height", function(d) {return d;});
```

**Sets the width of the rectangles to a fixed value**

**Sets the height equal to the value of the data associated with each element**

When we used the label values of our array to create `<div>` content with labels in section 2.2.1, we pointed to the object's `label` attribute. Here, because we're dealing with an array of number literals, we use the inline function to point directly at the value in the array to determine the height of our rectangles. The result, shown in figure 2.12, isn't nearly as interesting as you might expect.



**Figure 2.12 The default setting for any shape in SVG is black fill with no stroke, which makes it hard to tell when the shapes overlap each other.**

All the rectangles overlap each other—they have the same default x and y positions. The drawing is easier to see if the outline, or stroke, of your rectangles is different from their fill. We can also make them transparent by adjusting their opacity style, as shown in figure 2.13.



**Figure 2.13 By changing the fill, stroke, and opacity settings, you can see the overlapping rectangles.**

```
d3.select("svg")
  .selectAll("rect")
  .data([15, 50, 22, 8, 100, 10])
  .enter()
  .append("rect")
  .attr("width", 10)
  .attr("height", function(d) {return d;})
  .style("fill", "blue")
  .style("stroke", "red")
  .style("stroke-width", "1px")
  .style("opacity", .25);
```

You may wonder about practical use of the second variable in the inline function, typically represented as `i`. One use of the array position of a data value is to place visual